Table of contents

Simple Service Discovery Protoc...

External control service comma...

General ECP commands

Roku TV ECP commands

Show all...

External Control Protocol (ECP)

The External Control Protocol (ECP) enables a Roku device to be controlled over a local area network by providing a number of external control services. The Roku devices offering these external control services are discoverable using SSDP (Simple Service Discovery Protocol). ECP is a simple RESTful API that can be accessed by programs in virtually any programming environment.

As of Roku OS 14.1, the Settings > System > Advanced system settings > Control by mobile apps feature must be set to "Enabled" for a Roku device to receive the following ECP commands:

- keypress
- keydown
- keyup
- query/icon
- query/tv-channels
- query/tv-active-channel

In addition, the following ECP commands require the Roku device to be in developer mode (/docs/developer-program/getting-started/developer-setup.md) and the Control by mobile apps setting to be "Enabled":

- query/chanperf
- query/r2d2-bitmaps
- query/sgnodes
- query/sgrendezvous and sgrendezvous
- query/registry
- query/graphics-frame-rate
- query/fwbeacons and fwbeacons
- query/app-object-counts
- query/app-state
- exit-app

As of Roku OS 12.0, the "search" command is no longer available.

Support for sending ECP commands from within a Roku app has been discontinued. Apps may no longer include code in their app that is designed to issue any type of ECP command. Static Analysis testing (/docs/developer-program/dev-tools/static-analysis-tool/static-analysis-tool.md) has been updated to check apps for ECP commands. Apps that include ECP commands in their code will automatically be blocked from publishing to the Streaming Store.

In addition, ECP commands may not be sent from 3rd-party platforms (for example, mobile applications).

Apps may still include code for handling incoming ECP commands sent by the Roku OS for deep links (/docs/developer-program/discovery/implementing-deep-linking.md), voice controls (/docs/developer-program/media-playback/voice-controls/transport-controls.md), and so on.

To further leverage ECP commands for testing an app's performance and behavior, it is recommended that developers integrate Roku's automation test software (/docs/developer-program/dev-tools/automated-channel-testing/automated-testing-overview.md) in their

test suite.

Simple Service Discovery Protocol (SSDP)

SSDP is an industry IETF standard network protocol for discovery of local area network services. Roku devices advertise their external control services using the multicast SSDP so that programs can discover the IP address of Roku devices in the area. There is a standard SSDP multicast address and port (239.255.255.250:1900) used for local area network communication. The Roku device responds to M-SEARCH queries on this IP address and port.

To query for a Roku device IP address, send the following HTTP request to 239.255.255.250 port 1900:

```
M-SEARCH * HTTP/1.1
Host: 239.255.255.250:1900
Man: "ssdp:discover"
ST: roku:ecp
```

There *must* be a blank line at the end of the file above. If you put the above request into a file such as roku_ecp_req.txt, you can issue the following command on most Linux machines to test the request:

```
$ ncat -u 239.255.255.250 1900 < roku_ecp_req.txt
```

If you view the response using Wireshark, and filter on port 1900, you can see the Roku device response (Ncat has trouble receiving multicast traffic, so viewing the response using Ncat does not work). The response has the following format:

```
HTTP/1.1 200 OK
Cache-Control: max-age=3600
ST: roku:ecp
Location: http://192.168.1.134:8060/
USN: uuid:roku:ecp:P0A070000007
```

If you get a 200 status response, the Location header is valid. You can parse out the URL for the Roku device external control services from the Location header. The Roku device serial number is contained in the USN line after uuid:roku:ecp. Note that if there are multiple Roku devices in your local area network, you will get multiple responses. Your program could keep a map of USNs to location URLs, and allow the user to select which Roku device on the network to control. We recommend you let the user assign names to the USNs.

When parsing headers in the response, in accordance with the UPnP Device Architecture specification, field names should not be treated as case sensitive. That means that, for example, the Location header may begin with either "Location:" or "LOCATION:" or "location:", and so forth.

Please note the Cache-Control header. Roku devices multicast NOTIFY messages periodically (approximately every 20 minutes). It is safe to assume the unit is no longer available if you have not received a new NOTIFY message before the Cache-Control max-age time expires.

External control service commands

The external control services provided by ECP are included in a simple RESTful API accessed using HTTP on port 8060. Once you have the Roku device IP address, you can issue the following external control service commands to the Roku device.

General ECP commands

Command	Description
query/media-player	Returns a child element named 'player' that identifies the current stream segment and position of the conte format, and buffering. This command is accessed using
keydown/<KEY>	Equivalent to pressing the remote control key identifie POST with no body.
keyup/<KEY>	Equivalent to releasing the remote control key identifi POST with no body.
keypress/<KEY>	Equivalent to pressing down and releasing the remote command, and the keydown and keyup commands, to screen is active, as described in Keypress Key Values. T
query/device-info	Retrieves device information similar to that returned b GET. As of Roku OS 15.0, this command returns the followin control have been enabled on a Roku streaming player - supports-tv-power-control - supports-audio-volume-control
query/icon/<APP_ID>	supports-tv-power-control supports-audio-volume-co identified by appID. The binary data with an identifying accessed using an HTTP GET. Example: GET /query/icc

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Command	Description
query/chanperf query/chanperf/<*channelId*>?duration-seconds=<*seconds*>	Returns the current memory and CPU utilization of th bytes). The foreground app may either be a sideloadec results for an app in the app store, the device must be generate the package file. • Including the channelId option in the path output use this command, the device must be keyed with package file. The app's process ID (pid) is added tc
query/r2d2-bitmaps	Returns a list of the assets that have been loaded into maximum memory on your device (in bytes). As of Roku OS 11.5, this query returns all bitmaps in te attributed to an app.
query/sgnodes/all?count_only=true&sizes=true	Returns all the nodes created by the currently running node (held in the Roku platform) and bscref references • The bcsref count includes references from "m." va references do not increase bscref counts. The bsc of the resources that the app itself controls. - The count_only parameter returns the total number - The size parameter returns the memory used by the
query/sgnodes/roots?count_only=true&sizes=true	Prints every existing node without a parent that has b these un-parented nodes means they are being kept a variables local to a function, arrays, or associative arra field of a node.
query/sgnodes/nodes?node-id=*nodeId*&count_only=true&sizes=true	Prints nodes with an id field set to node_ID, except it, b down the whole list in the order of node creation. It wi

Command	Description
sgrendezvous	Lists the node rendezvous events for a sideloaded app account. Use the following commands to enable the logging of use these commands, the device must have developer

Command	Argument	Description
sgrendezvous/track (POST request)	channel_id (optional)	Starts the lo Only one app any queued r To track rend body: POST http:/ POST http:/ The response <sgrendezvo <tracking- <status>O </sgrendezv
query/sgrendezvous		Returns the r enabled, or si maximum of this limit are the total cou To retrieve re GET http:// See query/sg response.

Command	Description
query/registry/*<channelId>*	Lists the entries in the device registry for a sideloadec developer's account. The app ID must be provided; for
query/graphics-frame-rate *Available since Roku OS 12.0*	Returns the recent number of rendered graphics fram frame rate). Developer mode must be enabled to use t

Command	Argument	Description
sgrendezvous/untrack		To stop the t with no JSON POST http:/ The response <sgrendezvc <tracking- <status>O </sgrendezv

Command	Description
fwbeacons *Available since Roku OS 12.0*	Tracks app and media lifecycle events for a specific ap developer mode enabled. **Command** / **Description** fwbeacons/track fwbeacons/track/<*channelId*> (POST request) — Enables tracking When tracking i for retrieval with not queried. If t events on the p If the *channelId* foreground UI a All devices may monitor apps fr developer key. query/fwbeacons — Retrieves the a the previous que done. fwbeacons/untrack — Disables trackin discards all queu
query/app-object-counts/<*channelId*> *Available since Roku OS 13.0*	Returns the counts for the different BrightScript node counts of each type of object held by their Brightscrip The app may either be a sideloaded app or an app fror the app store, the device must be keyed with the sam file.
query/app-state/<*appId*> *Available since Roku OS 13.0*	Returns the current app state: "active", "background" (The app may either be a sideloaded app or an app fror the app store, the device must be keyed with the sam file. If the app is not installed, this command returns an er
exit-app (POST request) *Available since Roku OS 13.0*	Suspends or terminates an app that is running: • If the app supports Instant Resume and is runnin app (the app runs in the background). • If the app supports Instant Resume and is runnin Resume and is running, sending this command te

Command	Description
input	Sends custom events to the current application. It tak string URI parameters. The external control server pla passes them directly through to the currently executi roInput object. Input Command Conventions (/docs/developer-progra conventions) includes detailed recommendations on h Messages of type roInputEvent (/docs/references/brig method that will obtain the associative array. The argu This command is sent using an HTTP POST with no bo `POST /input?acceleration.x=0.0&acceleration.y=0.0&`

- Including duration-seconds in the query string executes and repeats the chanperf command the specified number of seconds. To cancel a repeating command, use the chanperf command with no arguments or with the duration-seconds parameter set to 0 ("chanperf" or "chanperf/duration-seconds=0"). The default duration is 1 second.
- The osref count also includes child references and references from Roku SceneGraph interface fields. For example, for any node with a parent, the parent will count as one osref on the child. Additionally, any field of type node, nodearray, or assocarray will add one osref to each node referenced from within that field. These could be in variables local to a function, arrays, or associative arrays, including a component global m or an associative array field of a node. The reported osref count may vary from release to release of Roku OS; the information here is provided only to give a sense of the kinds of items that the count includes.

Roku TV ECP commands

Roku TV devices additionally support the following external control services.

Command	Description
query/tv-channels	Returns information about the TV channel / line-up available for viewing in the TV tuner UI.
query/tv-active-channel	Returns information about the currently tuned TV channel.

Input command conventions

As the Roku OS simply marshals the arguments to the input command and passes them to the app script, the forms below compose a conventional way to communicate input from several common input device types.

Sensor input values

There are four sensor input values to report: accelerometer, orientation, gyroscope (rotation), and magnetometer (magnetic). All except orientation are vectors in a cartesian coordinate system relative to the device in its default orientation:

- +x = to the right of the front face of the device (usually the short side)
- +y = to the top of the front face of the device (usually the long side)
- +z = out of the front face of the device (toward the viewer)

The orientation coordinate system is relative to the point on the surface of the Earth between the device and the center of the Earth:

- +x = east
- +y = north
- +z = towards the center of the Earth (down)

The type in all such cases is a string representation of a signed floating point number, with or without an explicit decimal, and with or without a signed integer exponent following the letter E. A missing decimal will be presumed after the rightmost present digit, and a missing exponent will be presumed 0.

Accelerometer

indicates: acceleration in each dimension relative to free fall units: meters/sec^2 names: acceleration.x, acceleration.y, acceleration.z

Orientation

indicates: angular displacement from flat/level and true (or magnetic?) north. units: radians names: orientation.x, orientation.y, orientation.z notes: Accurate indication of this is not generally possible without correlation with other sensors or assumptions. Devices make assumptions to flip the display, for example, that assume that the device is usually not moving (much) so that all force is simply opposed to gravity, and that can be assumed to be the "up" direction. Deviation from magnetic north depends on a magnetometer, and deviation from true north also depends on geolocation.

Gyroscope

indicates: angular rotation rate about each axis using the right hand rule for sign units: radians/sec names: rotation.x, rotation.y, rotation.z

Magnetometer

indicates: magnetic field strength units: micro-Tesla names: magnetic.x, magnetic.y, magnetic.z

Touch and multi-touch

Touch and multi-touch commands take the same form. The resource is the same "input" as all other generic input commands.

Each action is decomposed to an argument in each dimension (of 2, x and y with the same orientation as for the sensor inputs, with origin in lower left). There is an additional "op" argument which can specify down, up, press (down and up), move, or cancel. Each input is also qualified with a pointer id that indicates the initial order of down touches in a multi-touch gesture.

Several such points can be specified in a single POST, especially a move, but a full triad of x, y, and op arguments should be sent, and expected for each point, within a POST that contains any of them.

Additional input values

Other information you might want to pass using the input command may include:

- sensor accuracy
- geolocation (from GPS)
- device-provided derivations of above sensor readings, for example "shake" from accelerometer, or "pinch" from multi-touch

External Control Protocol examples

The following are some example ECP commands sent via the curl command. The ROKU_DEV_TARGET environment variable should be set with the TCP/IP address of the target Roku device, e.g.

```
$ export ROKU_DEV_TARGET=192.168.1.134
```

Query/media-player example

The following command retrieves media player information.

```
$ curl "http://$ROKU_DEV_TARGET:8060/query/media-player"
```

The response includes the following fields:

```
<player error="false" state="play">
   <plugin bandwidth="44692475 bps" id="dev" name="MultiLive"/>
   <format audio="aac" captions="none" container="mp4" drm="none" video="mpeg4_15" video_res="1280
   <buffering current="1000" max="1000" target="0"/>
   <new_stream speed="128000 bps"/>
   <position>6916 ms</position>
   <duration>887999 ms</duration>
   <is_live>false</is_live>
   <runtime>887999 ms</runtime>
   <stream_segment bitrate="0" media_sequence="1" segment_type="mux" time="0"/>
</player>
```

Keypress example

The following command simulates a user hitting the "Home" button

```
$ curl -d '' "http://$ROKU_DEV_TARGET:8060/keypress/home"
```

Keyup/keydown example

The following commands move the cursor to the far left by holding down the Left key for 10 seconds

```
$ curl -d '' "http://$ROKU_DEV_TARGET:8060/keydown/left"
$ sleep 10
$ curl -d '' "http://$ROKU_DEV_TARGET:8060/keyup/left"
```

Query/device-info example

Below is an example query/device-info request and response.

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/docs/tools)

```
$ curl "http://$ROKU_DEV_TARGET:8060/query/device-info"
<device-info>
    <udn>015e5108-9000-1046-8035-b0a737964dfb</udn>
    <serial-number>1GU48T017973</serial-number>
    <device-id>1GU48T017973</device-id>
    <vendor-name>Roku</vendor-name>
    <model-number>4200X</model-number>
    <model-name>Roku 3</model-name>
    <model-region>US</model-region>
    <ui-resolution>1080p</ui-resolution>
    <supports-ethernet>true</supports-ethernet>
    <wifi-mac>b0:a7:37:96:4d:fb</wifi-mac>
    <ethernet-mac>b0:a7:37:96:4d:fa</ethernet-mac>
    <network-type>ethernet</network-type>
    <user-device-name>My Roku 3</user-device-name>
    <software-version>9.3.0</software-version>
    <software-build>09021</software-build>
    <secure-device>true</secure-device>
    <language>en</language>
    <country>US</country>
    <locale>en_US</locale>
    <time-zone>US/Pacific</time-zone>
    <time-zone-offset>-480</time-zone-offset>
    <power-mode>PowerOn</power-mode>
    <supports-suspend>false</supports-suspend>
    <supports-find-remote>false</supports-find-remote>
    <supports-audio-guide>false</supports-audio-guide>
    <developer-enabled>true</developer-enabled>
    <keyed-developer-id>70f6ed9c90cf60718a26f3a7c3e5af1c3ec29558</keyed-developer-id>
    <search-enabled>true</search-enabled>
    <voice-search-enabled>true</voice-search-enabled>
    <notifications-enabled>true</notifications-enabled>
    <notifications-first-use>false</notifications-first-use>
    <supports-private-listening>false</supports-private-listening>
    <headphones-connected>false</headphones-connected>
</device-info>
```

Query/icon example

This following command will return the icon for the app with ID 12 (Netflix). The response will be raw binary picture data, after HTTP headers, including one with the MIME type of the picture data.

```
$ curl -v "http://$ROKU_DEV_TARGET:8060/query/icon/12" -o image-12
< HTTP/1.1 200 OK
< Content-Length: 20679
< Cache-Control: no-cache
< Content-Type: image/jpeg
```

Query debugging examples

Query/chanperf example

The following command returns the current memory and CPU utilization of an app (RAM usage is reported in bytes).

```
curl "http://${ROKU_DEV_TARGET}:8060/query/chanperf"
```

The response includes the following fields:

```
<chanperf>
  <timestamp>1672980398506</timestamp>
  <plugin>
    <cpu-percent>
      <duration-seconds>1.000000</duration-seconds>
      <user>12.2</user>
      <sys>5.5</sys>
    </cpu-percent>
    <memory>
      <used>87785472</used>
      <res>87785472</res>
      <anon>24027136</anon>
      <swap>0</swap>
      <file>24727552</file>
      <shared>39030784</shared>
    </memory>
    <id>dev</id>
    <unsecured>
      <process-id>6861</process-id>
    </unsecured>
  </plugin>
  <status>OK</status>
</chanperf>
```

Query/r2d2-bitmaps example

The following command returns a list of the assets that have been loaded into texture memory, and the amount of used, available, and maximum memory on your device (in bytes).

```
curl "http://${ROKU_DEV_TARGET}:8060/query/r2d2-bitmaps"
```

The response includes the following fields:

(https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

```
<r2d2-bitmaps>
  <rographics>
    <sytem-memory>
      <used>115200</used>
    </sytem-memory>
    <texture-memory>
      <used>26841088</used>
      <available>93158912</available>
      <max>120000000</max>
    </texture-memory>
    <bitmap>
      <width>1920</width>
      <height>1080</height>
      <bpp>3</bpp>
      <size>8355840</size>
      <name>/tmp/plugin/CMAAAA2AIKa9/pkg:/images/splash-screen_fhd.jpg</name>
    </bitmap>
    <bitmap>
      <width>1920</width>
      <height>1080</height>
      <bpp>3</bpp>
      <size>8355840</size>
      <name>/nvram/theme/FHD/BackgroundBitmap.jpg</name>
    </bitmap>
  </rographics>
  <status>OK</status>
</r2d2-bitmaps>
```

Query/sgnodes/all example

The following command returns each existing node created by the currently running app. This includes the number of osref references to the node (held in the Roku platform) and bscref references (held in the app)..

```
curl "http://${ROKU_DEV_TARGET}:8060/query/sgnodes"
```

The response includes the following fields:

```
<sgnodes>
  <All_Nodes>
    <Default children="0" focusable="false" focused="false" index="0" name="" opacity="100" thread="rend
    <MainScene _sn="1" bounds="\{0, 0, 1920, 1080}" bscref="1" children="0" extends="Scene" focusable="
      <Poster _sn="2" bounds="\{0, 0, 1920, 1080}" bscref="0" inheritParentOpacity="false" inheritParentT
    </MainScene>
    <Node _psn="1" _sn="9" bscref="1" osref="1" rcid="0" />
    <LayoutGroup _psn="1" _sn="3" bounds="\{50, 50, 631, 536}" bscref="0" children="2" osref="1" rcid="0"
    <RenderableNode _psn="3" _sn="4" bounds="\{0, 0, 520, 440}" bscref="0" children="3" name="posterGr
    <Poster _psn="4" _sn="5" bounds="\{0, 0, 320, 240}" bscref="0" loadStatus="3" osref="1" rcid="0" uri="p
    <Poster _psn="4" _sn="6" bounds="\{100, 100, 320, 240}" bscref="0" loadStatus="3" osref="1" rcid="0" t
    <Poster _psn="4" _sn="7" bounds="\{200, 200, 320, 240}" bscref="0" loadStatus="3" osref="1" rcid="0" t
    <Label _psn="3" _sn="8" bounds="\{0, 490, 631, 46}" bscref="0" color="#ffff00ff" osref="1" rcid="0" text
  </All_Nodes>
  <status>OK</status>
</sgnodes>
```

query/sgrendezvous example

The following commands enable rendezvous tracking and list the rendezvous events for a sideloaded app or production/beta app linked to the Roku developer's account:

```
$ curl -d " "http://$ROKU_DEV_TARGET:8060/query/sgrendezvous/track"
$ curl "http://${ROKU_DEV_TARGET}:8060/query/sgrendezvous"
```

The response includes an element for each node rendezvous event that was logged. Each event recorded is one that occurred after tracking was enabled or after the previous call to query/sgrendezvous, whichever occurred last.

- The end-tm and start-tm fields indicate the number of milliseconds that elapsed during the rendezvous.
- The timestamp indicates the time that the query was executed.

(https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

```
<sgrendezvous>
  <data>
    <tracking-enabled>true</tracking-enabled>
    <plugin-id>dev</plugin-id>
    <drop-count>0</drop-count>
    <count>4</count>
    <item>
      <id>471</id>
      <start-tm>2731136</start-tm>
      <end-tm>2731136</end-tm>
      <line-number>21</line-number>
      <file>pkg:/components/ServiceTask.brs</file>
  </data>
  <timestamp>1656713004102</timestamp>
  <status>OK</status>
</sgrendezvous>
```

The following command disables rendezvous tracking:

```
$ curl -d '' "http://$ROKU_DEV_TARGET:8060/query/sgrendezvous/untrack"
```

query/registry example

The following command returns the registry entries for the app.

```
curl '' "http://$ROKU_DEV_TARGET:8060/query/registry/dev"
```

The response includes the following fields:

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

```
<plugin-registry>
  <registry>
    <dev-id>e090ac01d342483bb28831a7e1afff8e</dev-id>
    <plugins>dev</plugins>
    <space-available>9168</space-available>
    <sections>
      <section>
        <name>UserInfo</name>
        <items>
          <item>
            <key>NextPaymentDate</key>
            <value>2022-09-17T17:17:55</value>
          </item>
          <item>
            <key>UserId</key>
            <value>1429492</value>
          </item>
        </items>
      </section>
    </sections>
  </registry>
  <status>OK</status>
</plugin-registry>
```

query/fwbeacons example

The following commands enable app and media lifecycle event tracking and list the events for a sideloaded app:

```
$ curl -d '' "http://${ROKU_DEV_TARGET}:8060/fwbeacons/track/dev"
$ curl "http://${ROKU_DEV_TARGET}:8060/query/fwbeacons"
```

The response includes the following fields:

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

```
<fwbeacons>
  <tracking-enabled>true</tracking-enabled>
  <plugin-id>dev</plugin-id>
  <dropped-count>0</dropped-count>
  <interval-drop-count>0</interval-drop-count>
  <count>6</count>
  <app-compile-initiate>
    <timestamp>1675727290681</timestamp>
  </app-compile-initiate>
  <app-compile-complete>
    <timestamp>1675727290706</timestamp>
  </app-compile-complete>
  <app-splash-initiate>
    <timestamp>1675727290733</timestamp>
  </app-splash-initiate>
  <app-splash-complete>
    <timestamp>1675727291615</timestamp>
  </app-splash-complete>
  <app-exit-initiate>
    <timestamp>1675727295248</timestamp>
  </app-exit-initiate>
  <app-exit-complete>
    <timestamp>1675727295333</timestamp>
  </app-exit-complete>
  <timestamp>1675727301787</timestamp>
  <status>OK</status>
</fwbeacons>
```

query/app-object-counts example

The following command returns the counts for the different BrightScript node objects in the app.

```
$ curl " "http://$ROKU_DEV_TARGET:8060/query/app-object-counts/<channelId>"
```

The response includes the following fields:

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

```
<?xml version="1.0" encoding="UTF-8" ?>
<app-object-counts>
  <channel-id>581251</channel-id>
  <channel-title>Roku Developers</channel-title>
  <channel-version>9.3.10</channel-version>
  <objects>
    <object>
      <name>roArray</name>
      <count>100</count>
    </object>
    <object>
      <type>roAssociativeArray</type>
      <count>200</count>
    </object>
    <object>
      <name>roSGNode</name>
      <subtype>Label</subtype>
      <count>300</count>
    </object>
    <object>
      <name>roSGNode</name>
      <subtype>ContentNode</subtype>
      <count>400</count>
    </object>
    <object>
      <name>roSGNode</name>
      <subtype>AppScene</subtype>
      <count>1</count>
    </object>
    <object>
      <name>roSGNode</name>
      <subtype>Poster</subtype>
      <count>200</count>
    </object>
    <object>
      <type>roString</type>
      <count>2000</count>
    </object>
  <objects>
  <<objects-count>>3201</<objects-count>>
</app-object-counts>
```

query/app-state example

The following command returns the state of the app state: "active", "background" (suspended; running in the background), or "inactive".

```
$ curl " "http://$ROKU_DEV_TARGET:8060/query/app-state/<appId>"
```

The response includes the following fields:

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

```
<?xml version="1.0" encoding="UTF-8" ?>
<channel>
    <channel-id>581251</channel-id>
    <channel-state>active</channel-state>
      <channel-version>1.2</channel-version>
</channel>
```

exit-app example

The following command suspends or terminates an app that is running.

```
$ curl -d '' "http://$ROKU_DEV_TARGET:8060/exit-app/<channelId>"
```

This command returns an HTTP 200 success code if the app was active or suspended. It returns an HTTP 404 (Not Found) error if the app was not running.

Input examples

The following command passes three components of acceleration through to the app. All query string parameters are passed to the currently running app. The remote app and the currently running app just need to agree on the query string parameters and any communication can be developed.

```
$ curl -d '' "http://$ROKU_DEV_TARGET:8060/input?acceleration.x=0.0&acceleration.y=0.0&acceleration
```

The following command indicates that a touch at the given x and y has touched down on the screen.

```
$ curl -d '' "http://$ROKU_DEV_TARGET:8060/input?touch.0.x=200.0&touch.0.y=135.0&touch.0.op=down
```

Query/tv-channels example

Below is an example of the Roku TV query/tv-channels response.

```
$ curl "http://$ROKU_DEV_TARGET:8060/query/tv-channels"

<tv-channels>
  <channel>
    <number>1.1</number>
    <name>WhatsOn</name>
    <type>air-digital</type>
    <user-hidden>false</user-hidden>
  </channel>
  <channel>
    <number>1.3</number>
    <name>QVC</name>
    <type>air-digital</type>
    <user-hidden>false</user-hidden>
  </channel>
</tv-channels>
```

Query/tv-active-channel example

Below is an example of the Roku TV query/tv-active-channel response.

```
$ curl "http://$ROKU_DEV_TARGET:8060/query/tv-active-channel"

<tv-channel>
  <channel>
    <number>14.3</number>
    <name>getTV</name>
    <type>air-digital</type>
    <user-hidden>false</user-hidden>
    <active-input>true</active-input>
    <signal-state>valid</signal-state>
    <signal-mode>480i</signal-mode>
    <signal-quality>20</signal-quality>
    <signal-strength>-75</signal-strength>
    <program-title>Airwolf</program-title>
    <program-description>The team will travel all around the world in order to shut down a global crime
    <program-ratings>TV-14-D-V</program-ratings>
    <program-analog-audio>none</program-analog-audio>
    <program-digital-audio>stereo</program-digital-audio>
    <program-audio-languages>eng</program-audio-languages>
    <program-audio-formats>AC3</program-audio-formats>
    <program-audio-language>eng</program-audio-language>
    <program-audio-format>AC3</program-audio-format>
    <program-has-cc>true</program-has-cc>
  </channel>
</tv-channel>
```

Deep linking to an app

One of the most common ways that app developers encounter ECP is deep linking. Deep linking lets other parts of the system launch a particular piece of content via parameters passed to the app at launch time. See this section on Deep Linking (/docs/developer-program/discovery/implementing-deep-linking.md) for implementation details.

The standard for deep linking uses parameters:

Parameter	Description	Possible Values
contentID	Partner defined unique identifier for a specific piece of content	Any value < 255 characters long and not using "&" or other characters that are not URL encoded
mediaType	Parameter to give context to the type of contentID passed	"series", "season", "episode", movie", "shortFormVideo", and "tvSpecial"

You trigger deep linking by doing an HTTP post to port 8060 on your Roku device. The general form is

```
http://<IP of Roku>:8060/launch/[dev | channelID]?contentId=<content ID>&MediaType=<mediaType>
```

The first example will launch the current sideloaded application and deep link to a season contentID 1234. Notice the -d ' ' which forces it to do a http post.

```
curl -d '' "http://$ROKU_DEV_TARGET:8060/launch/dev?contentID=13234&MediaType=season"
```

- You can also just launch the app without deep linking by removing the arguments from the URL:
  - http://$ROKU_DEV_TARGET:8060/launch/8378 (http://$ROKU_DEV_TARGET:8060/launch/8378)
- You can find the app ID of a production app by using one of the two deep link test tools
  - https://devtools.web.roku.com/DeepLinkingTester/ (https://devtools.web.roku.com/DeepLinkingTester/)
  - https://my.roku.com/account/add?channel=KX3UPK (https://my.roku.com/account/add?channel=KX3UPK)

Keypress key values

When the current screen on the Roku box includes an on-screen keyboard, any keyboard character can be sent via the keyup, keydown, and keypress commands. The key parameter can either be a key name, such as the name of a button on a remote control, or a printable character value

specified with the prefix "Lit_".

Printable ASCII character code values can be transmitted "as-is" with the "Lit_" prefix. For example, you can send 'a' 'r' with "Lit_r". In addition, any UTF-8 encoded character can be sent by URL-encoding it. For example, the euro symbol can be sent with "Lit_%E2%82%AC".

There are even some keys you can send that are not available on any physical remote. *Enter* is for completing keyboard entry fields, such as search fields (it is *not* the same as Select). *Search* is used for pressing and holding down the microphone/magnifying glass button, which causes the Roku Voice heads-up display to listen for a voice command.

The following are the key names that are recognized by ECP:

```
Home
Rev
Fwd
Play
Select
Left
Right
Down
Up
Back
InstantReplay
Info
Backspace
Search
Enter
```

Roku devices that support the "Find Remote" support:

```
FindRemote
```

*Note that query/device-info includes a supports-find-remote flag that indicates whether the Roku device supports FindRemote.

However, this does not specifically indicate that the device has a paired remote that supports "Find remote" as well.

Some Roku devices, such as Roku TVs, also support:

```
VolumeDown
VolumeMute
VolumeUp
PowerOff
```

Roku TV devices also support changing the app when watching the TV tuner input:

Roku (https://developer.roku.com/develop) Developers

(https://developer.roku.com/develop)

```
ChannelUp
ChannelDown
```

Roku TV devices also support keys to set the current TV input UI:

```
InputTuner
InputHDMI1
InputHDMI2
InputHDMI3
InputHDMI4
InputAV1
```

Example: On the on-screen keyboard, the string 'roku' can be sent via the following commands:

```
$ curl -d '' "http://$ROKU_DEV_TARGET:8060/keypress/Lit_r"
$ curl -d '' "http://$ROKU_DEV_TARGET:8060/keypress/Lit_o"
$ curl -d '' "http://$ROKU_DEV_TARGET:8060/keypress/Lit_k"
$ curl -d '' "http://$ROKU_DEV_TARGET:8060/keypress/Lit_u"
```

Example programs

There is a sample External Control Protocol application available that requires only glibc to compile. The program is self contained in the /examples/rokuExternalControl.c file in the sample. You can compile and run it with the following commands:

```
$ cd SDK_directory
$ gcc ./examples/rokuExternalControl.c -o rokuExternalControl
$ ./rokuExternalControl
```

On Windows, it can be compiled with the following line:

```
> cl /D "WIN32" rokuExternalControl.c
```

The program first uses SSDP to query for Roku devices in the local area network. The first Roku device that responds is the one to which commands are sent. All requests and responses are printed to stdout so that you can easily follow what the program is doing. The program next moves the cursor to the home screen and highlights the "Netflix" program. It does this by sending the Home key command, and then holding down the left key for twelve seconds. Then it sends the Right key three times. After keeping the "Netflix" program highlighted for five seconds, the program launches the "simplevideoplayer" application with url and streamformat as keys in the associative array passed to the Main() entry point. The simplevideoplayer application immediately launches the roVideoScreen when launched with an associative array containing valid url and streamformat keys.

With ECP, you have complete control of your Roku device over the network. We can't wait to see what kind of solutions our developer community can create. The sample C program can be quickly modified to run in a variety of environments including Firefox, IE, and other browser plugins, iPhone, iPad, and other mobile device environments.

The SDK also includes a couple of Java applications. There is an Android remote application in examples/source/ecp_client/android_remote, and a simple application to find Roku devices on the local area network in examples/source/ecp_client/Roku_Finder.

DIAL (Discovery and Launch)

The Roku platform supports the DIAL (Discovery and Launch) protocol. DIAL is a simple network protocol for discovering first screen devices, and applications from a second screen (such as a mobile iOS or Android application,) and for launching first screen applications on the first screen device from the second screen app. In the context of the Roku platform, the first screen device is the Roku device itself. A first screen application is a DIAL-aware app installed on the Roku device. Complete details of the DIAL specification can be found here:

http://www.dial-multiscreen.org/dial-protocol-specification (http://www.dial-multiscreen.org/dial-protocol-specification).

Many current Roku developers are familiar with the Roku external control protocol (ECP) which includes functionality similar to DIAL. An experienced Roku developer may thus fairly ask the question "why do I need DIAL?" One reason is that you may already have a DIAL based second screen implementation for use with other platforms. DIAL support on the Roku platform means that you don't need to add a second protocol to your current application for discovery and launch.

The Roku DIAL sample (https://github.com/rokudev/samples/tree/master/utilities) contains detailed documentation of Roku DIAL support, as well as BrightScript, Android, and iOS sample applications. In DIAL parlance, the BrightScript sample is the first screen application, and the Android and iOS apps are the second screen applications. These sample applications should help you get started with your own Roku DIAL support.

Previous Next

About ads & cookies (https://docs.roku.com/published/cookiepolicy)

My Privacy Choices
(https://privacy.roku.com/ccpa)
Developer Terms & Agreements (https://developer.roku.com/docs/features/legal/developer-terms.md)

CA B2B Privacy Notice

© 2008-2026 Roku, Inc. (https://www.roku.com)